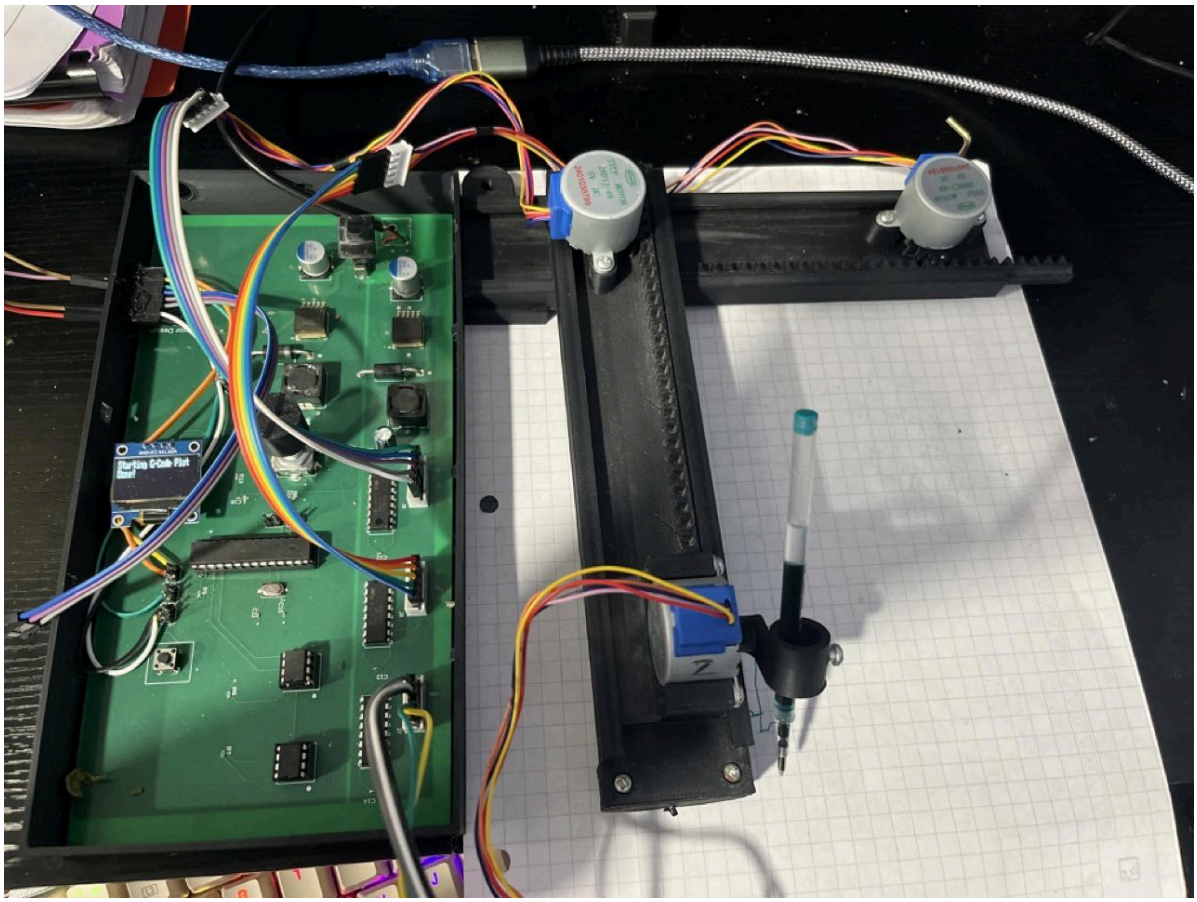
Design and Implementation of a 2D Plotter Using Unipolar Stepper Motors (2024)

Angel Barrera John San Juan Ranbir Saini



[1]

***Abstract*—This paper presents the design and implementation of a two-axis stepper motor plotter (2D plotter), a compact and cost-effective device capable of rendering vector-based drawings on a 2D surface. The system utilizes three unipolar stepper motors controlled by an ATmega328P microcontroller to achieve precise motion in the X, Y, and Z axes. Movement calibration is achieved through step-per-millimeter constants, while a pen-lift mechanism enables selective contact with the drawing surface. Commands are processed over a serial interface, allowing the plotter to interpret basic G-code-like instructions for coordinated motion. Emphasis is placed on mechanical accuracy, motor synchronization, and command parsing for reliable operation. The design serves as a foundational platform for educational, artistic, and prototyping applications. Keywords include stepper motor control, embedded systems, pen plotter, and G-code parsing.**

***Index Terms*—**
Axis control
Coordinate mapping
Firmware logic
G-code
Mechanical actuation
Microcontroller
Microstepping
Motion sequencing
Pen lift mechanism
Plotting accuracy
Serial command interface
Signal timing
Stepper motor calibration
Stepper motor control
Vector-based plotting

I. INTRODUCTION

This project involved designing and building a 2D plotter to gain hands-on experience with stepper motor control, embedded programming, and mechanical system integration. The plotter uses three unipolar stepper motors: two for X and Y axis movement, and one for pen up/down control. An ATmega328 microcontroller runs the control logic, and ULN2003A driver boards are used to interface with the motors.

The purpose of this project was to learn how to control multiple stepper motors using programmed step sequences and translate coordinate-based inputs into physical motion. Basic commands are processed over serial to move the pen to specific positions on the drawing surface.

This setup provides a simple example of how embedded systems can be used to automate mechanical tasks. The main learning goals were understanding motor driving techniques, timing and synchronization, and how mechanical limitations affect accuracy and performance.

This project focuses on building a 2D plotter that can move a pen along the X and Y axes to draw shapes based on digital input. To make this possible, we divided the system into several key subsystems: motor control, power delivery, axis calibration, EEPROM memory for saving settings, a 0.9-inch OLED screen for displaying system status, and a rotary encoder for manual navigation and input. Each component plays a role in ensuring the plotter operates accurately and is user-friendly. Our team split these tasks, and the following sections explain what each member contributed—starting with motor setup, power management, and movement calibration.

---

[1]

This work was supported in part by **State University City Bay** as part of CMPE 493 (Corresponding authors: [Angel Barrera, John San-Juan, Ranbir Saini]).

Angel Barrera, John San-Juan, and Ranbir Saini are students of the Department of Computer Engineering, California State University East Bay, Hayward, CA 94542 USA (e-mail: abarrera21@horizon.csueastbay.edu, jsanjuan2@horizon.csueastbay.edu, rsaini11@horizon.csueastbay.edu).

II. System Overview

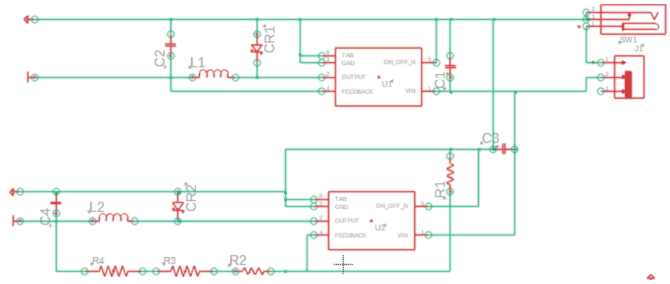
The 2D plotter system is built around several core subsystems working together to interpret user commands and produce accurate, repeatable motion across two axes. At the heart of the system is an ATmega328PU microcontroller, responsible for coordinating all hardware components.

The X, Y, and Z motion is driven by three unipolar stepper motors, each connected to a ULN2003A driver module. The X and Y motors move the pen across the drawing area, while the Z motor controls pen lift. The system receives movement instructions through a serial interface and translates them into stepper motor signals.

To aid user interaction, the system includes a 0.9-inch OLED screen that displays live feedback such as current coordinates, mode selection, or system status. A rotary encoder allows the user to navigate menu options, confirm selections, and manually adjust the pen position when needed.

An external EEPROM (AT24C256) is used to store configuration settings such as steps-per-millimeter, home positions, and previous drawing modes. This allows the plotter to retain critical data between power cycles.

Power is delivered through a buck converter, which ensures consistent voltage to both logic-level components and motor drivers. Proper calibration of steps-per-millimeter and synchronized motor control are essential for accurate plotting, and the system is designed to accommodate future expansions such as SD card integration or Bluetooth serial control.



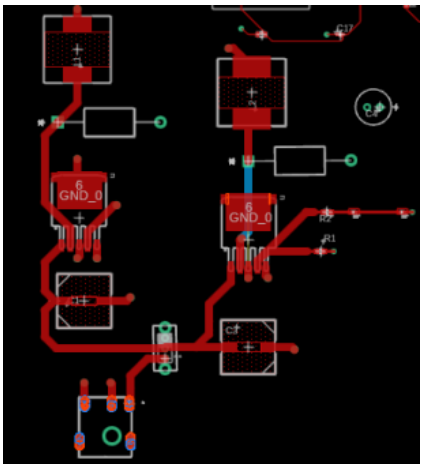
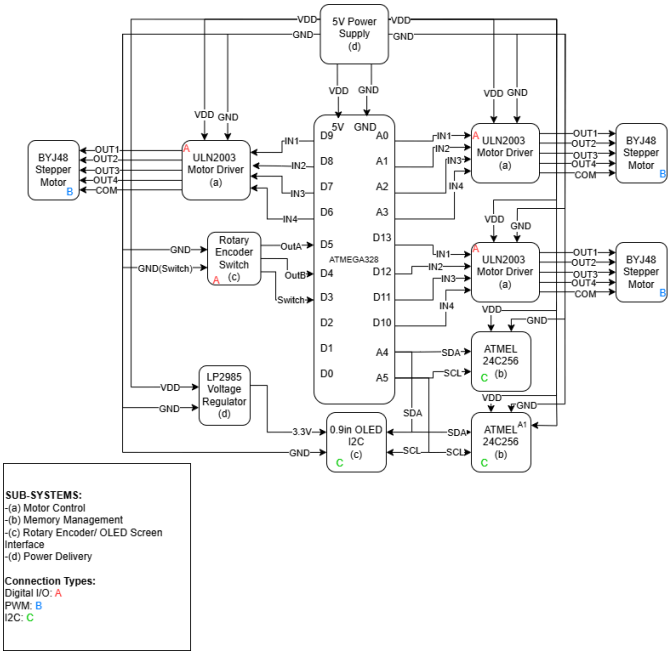


Fig. 2 illustrates the overall block diagram of the system architecture.

Figure 2:



A. Abbreviations and Acronyms

EEPROM – Electrically Erasable Programmable Read-Only Memory

G-code – Geometric Code (used in numerical control systems)

GUI – Graphical User Interface

I²C – Inter-Integrated Circuit (also referred to as I2C)

IDE – Integrated Development Environment

IO – Input/Output

MCU – Microcontroller Unit

OLED – Organic Light-Emitting Diode

PCB – Printed Circuit Board

PWM – Pulse Width Modulation

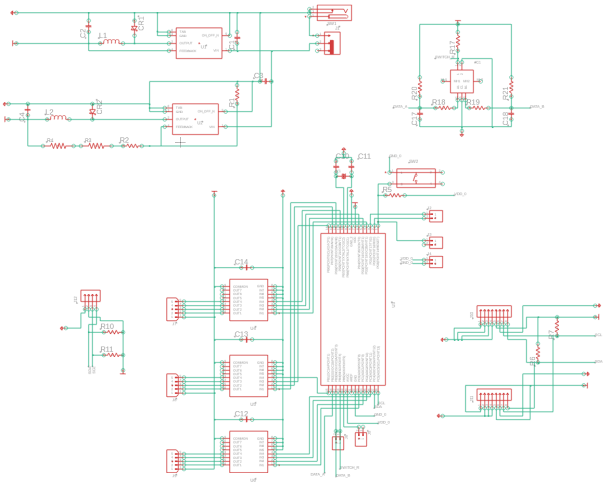
SCL – Serial Clock Line (I²C)

SDA – Serial Data Line (I²C)

III. Full Schematics

The schematic diagrams represent the core electrical design of the 2D plotter and provide a complete blueprint for replicating or modifying the system. Every component—ranging from the microcontroller to the stepper motor drivers and user interface devices—is laid out with clear interconnections to ensure compatibility, minimize noise, and maintain functional stability. This section presents the finalized schematics used during development, along with rationale behind key design choices such as power routing, pin assignments, communication protocols, and protection elements. These schematics are critical not only for assembling the system but also for debugging, future scaling, or redesigning individual

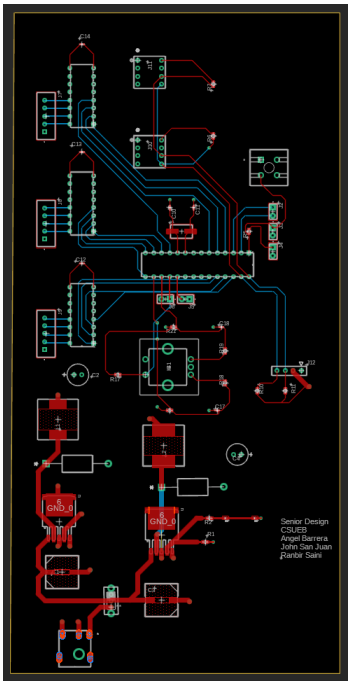
subsystems.



Full Schematics

IV. Layout

The PCB layout was designed with a strong emphasis on signal integrity, power stability, and manufacturability. Translating the schematic into a real-world board required careful placement of components to minimize trace lengths, avoid crosstalk, and ensure proper grounding. High-current paths for the stepper motors were isolated from the microcontroller logic to reduce noise and prevent brownout conditions. Attention was also given to component orientation and labeling to simplify debugging and assembly. This section outlines the physical layout strategy, layer stackup, trace routing, and spacing rules used to optimize both performance and reliability.



Full PCB Layout

V. Microcontroller Unit

The microcontroller used in this project is the ATmega328P, which is the same chip found on standard Arduino Uno boards. It was chosen due to its low cost, ease of programming, and compatibility with common development tools like the Arduino IDE. The ATmega328P operates at 16 MHz and provides enough digital I/O pins to control all three stepper motors and handle serial communication.

In this setup, digital output pins are connected to the ULN2003A stepper motor driver inputs. The microcontroller sends a series of HIGH and LOW signals in a specific pattern to step the motors in the desired direction. Each axis uses four pins to control one motor, and the Z-axis (pen up/down) is also controlled the same way.

Currently all stepper motors are controlled using the built-in Stepper library, which outputs a step sequence to four digital pins for each motor.

The three unipolar stepper motors are each connected to a ULN2003A driver board, and the pin assignments on the ATmega328P are shown in Table I. The X and Y motors control horizontal and vertical movement, respectively, while the Z motor handles the pen up/down motion.

The Stepper class is instantiated using four control pins per motor, and each motor is set to 2048 steps per revolution. The firmware is structured to respond to basic movement commands over serial, such as "X100" or "PD", and convert them into discrete motor steps. The Z motor moves the pen

using fixed values defined by PEN_DOWN_STEPS and PEN_UP_STEPS

Table I shows the pin connections between the ATmega328P and the ULN2003A drivers for each axis.

Table I:

Axis	UNL2003A Inputs	ATMega 328P Pins
X-Axis	IN1–IN4	D6, D7, D8, D9
Y-Axis	IN1–IN4	A0, A1, A2, A3
Z-Axis	IN1–IN4	D2, D3, D4, D5
OLED display		A4 (SDA), A5 (SCL)
EEPROM chip 1		A4 (SDA), A5 (SCL)
EEPROM chip 2		A4 (SDA), A5 (SCL)
Rotary encoder switch		D12

A. Equations

The following equations define step conversion, delta movement, and the parametric equations used for circular plotting.

(1) $S_x := \text{round}(x \cdot \text{stepsPerMM}_x)$

(2) $S_y := \text{round}(y \cdot \text{stepsPerMM}_y)$

(3) $\Delta S := S_{target} - S_{current}$

(4) $\theta := \frac{2\pi i}{N}$

(5) $x := x_c + r \cdot \cos(\theta)$

(6) $y := y_c + r \cdot \cos(\theta)$

B. Algorithms

Algorithm 1 Finite State Machine for Mode Control

Initialize enum OpMode { IDLE, MANUAL_CONTROL, PLOTTING }

loop():

if (currentMode == IDLE): wait for user input

else if (currentMode == MANUAL_CONTROL):

accept and execute manual axis commands

else if (currentMode == PLOTTING):

read and execute G-code from EEPROM

The FSM controls the plotter's behavior across three operational states. The system switches modes based on parsed serial commands and enables different features depending on the current mode.

Algorithm 2 Manual Command Parsing

if (command starts with 'X', 'Y', or 'Z'):

axis ← first character of command

steps ← parse remaining characters as integer

if axis == 'X': stepperX.step(steps)

if axis == 'Y': stepperY.step(steps)

if axis == 'Z': stepperZ.step(steps)

This algorithm parses manual movement commands such as X100 and moves the respective motor accordingly. It allows users to manually control motion for calibration or positioning.

Algorithm 3 G-Code Parsing from EEPROM

while (readLineFromEEPROM_to_Buffer(line)):

if (line starts with 'G'):

extract command number

extract X, Y, Z parameters (if present)

constrain values to plot area

moveToZ(Z) if Z is found

moveToXY(X, Y) if X or Y found

This algorithm reads and executes stored G-code lines. It supports G0, G1, G90, and G91 commands, allowing for automated plotting from EEPROM memory.

Algorithm 4 EEPROM Line Reader

initialize buffer

```
while (eepromReadAddress < TOTAL_MEMORY):
    byte ← readEEPROMByte(eepromReadAddress)
    if byte == END_OF_DATA_MARKER or byte ≥ 0xE0:
        return false
    if byte == '\n':
        null-terminate buffer
        return true
    append byte to buffer
    increment eepromReadAddress
```

Algorithm 5 XY Coordinate Step Interpolation

```
stepsX ← targetStepsX - currentStepsX
stepsY ← targetStepsY - currentStepsY

while (stepsDoneX < abs(stepsX) or stepsDoneY < abs(stepsY)):
    if (stepsDoneX < abs(stepsX)):
        stepperX.step(directionX)
        stepsDoneX++
    if (stepsDoneY < abs(stepsY)):
        stepperY.step(directionY)
        stepsDoneY++
```

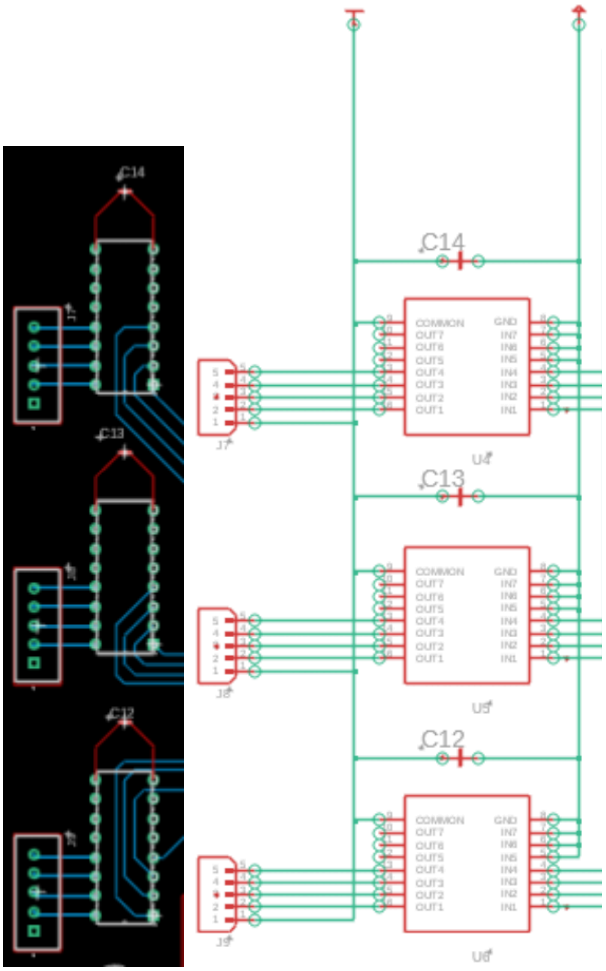
This performs basic two-axis interpolation, allowing the plotter to move diagonally by alternating steps on both axes. It doesn't implement smooth interpolation but is effective for low-complexity motion.

Algorithm 6 EEPROM Address Mapping

```
if (memoryAddress < EEPROM_SIZE):
    i2cAddress ← EEPROM_ADDR1
    chipAddress ← memoryAddress
else:
    i2cAddress ← EEPROM_ADDR2
    chipAddress ← memoryAddress - EEPROM_SIZE
```

This maps logical EEPROM addresses to physical memory locations across two EEPROM chips. It allows seamless access to a larger address space without manual chip selection.

VI. Motion and Control Interpolation



A. Motor Movement

The firmware supports both direct step-based commands (e.g., `X100`, `Y-50`) and coordinate-based plotting routines (e.g., `CIRCLE`, `SQUARE`), ensuring flexible motor control. To ensure all commands remain within the safe physical bounds of the machine, constrained position values are first calculated and then converted into stepper motor instructions.

Stepper movement is executed sequentially per axis to reduce complexity and avoid loss of position, with each axis updating its logical state after every move. The pen-lifting mechanism is also handled by a stepper motor (Z-axis), which toggles between drawing and non-drawing positions using fixed upward or downward movements.

The subsequent sections detail the equations and algorithms used to:

- Convert distances to motor steps,
- Constrain movement within the allowed drawing area,
- Compute coordinates for drawing primitives (lines, squares, and circles),

B. G-Code

G-code functionality is currently being tested by manually sending single-line instructions over the serial interface to the PCB. Commands such as X100, Y-50, and Z200 directly move the motors by a specified number of steps, while PD and PU control the pen's vertical movement. Higher-level instructions like CIRCLE and SQUARE trigger pre-programmed drawing routines.

When G-code movement commands are processed, position values are converted into step counts using equations (1) and (2), where the X and Y distances are multiplied by their respective steps-per-millimeter constants and rounded to whole steps. For drawing curves such as circles, each point on the path is calculated using a parametric representation. Equations (4), (5), and (6) define this logic: angle θ is incremented evenly based on the number of segments N (Eq. 4), and each target coordinate xi and yi is computed using cosine and sine functions around a center point (Eqs. 5 and 6). These positions are then converted into step differences (Eq. 3) and fed to the motors.

So far, commands are sent one at a time, allowing real-time testing of motor responses and calibration. Full G-code file parsing from EEPROM is planned, which will allow buffered execution of entire plots using the same equations.

C. Draw Area Size

The maximum effective plotting area of the system is limited to **8 cm × 8 cm** (80 mm × 80 mm). This constraint is dictated by the mechanical travel range of the stepper motors and the physical size of the plotting frame.

To prevent mechanical overtravel and ensure precise positioning within the 80 mm × 80 mm draw area, all target coordinates are bounded using the constrain() function.

These functions limit the effective input values for x and y to the calibrated range of the machine. If any G-code or manual movement command attempts to move beyond the physical area (e.g., X100 or Y-10), it is automatically adjusted back within range.

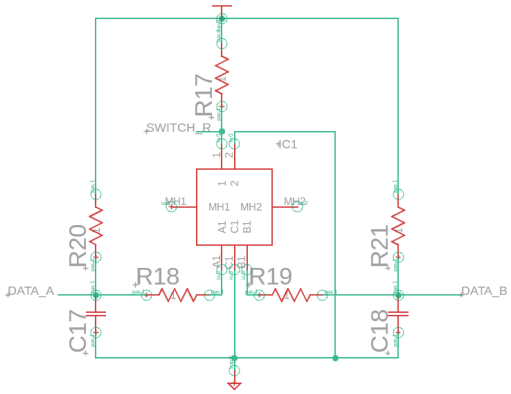
After constraining the position, the values are converted into motor steps using the previously defined step conversion equations (1) and (2).

VII. User Interface

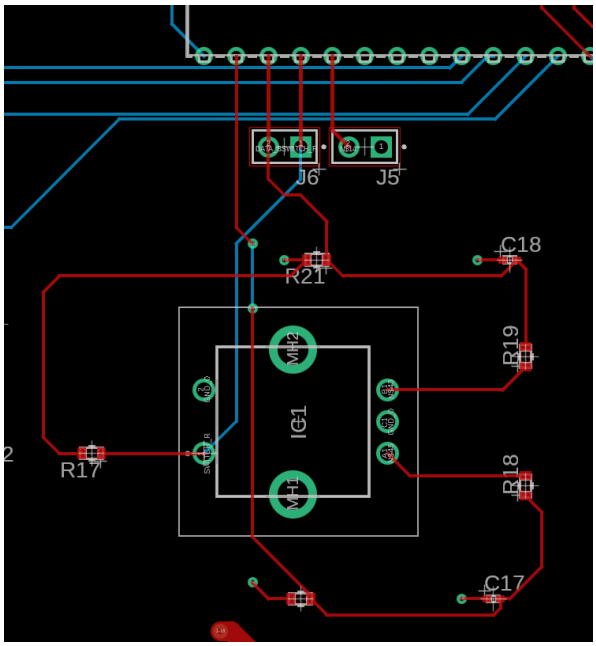
The user interface of the 2D-Plotter was designed to be simple and easy to use. It includes a rotary encoder and an OLED screen that allow users to select and start a drawing through different predefined g-codes.

A. Rotary Encoder

The rotary encoder is used to scroll through different g-code design and menu options. It has two main outputs (signal A and signal B) that send signals to the microcontroller when the knob is turned. The direction of rotation (clockwise or counterclockwise) is determined by checking the timing between these two signals. There is also a push-button built into the encoder that is used to make a selection.



Rotary Encoder Schematic



Rotary Encoder PCB Layout

To make sure the encoder works correctly, the following were implemented as per datasheet:

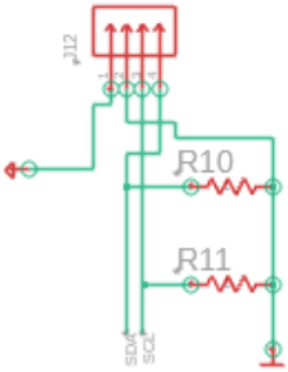
a) Pull-up resistors (10 kΩ) were added to keep signals stable when the encoder is not moving.
b) Capacitors (10 nF) were used to filter out electrical noise caused by the mechanical contacts inside the encoder.
c) A small software delay (5 milliseconds) was added between signal checks to reduce false readings when the knob moves too fast or skips [2], [3].

To track the rotary encoder's rotation direction, the code monitors changes in the A signal, checking for a `HIGH` to `LOW` transition. When this happens, it checks the B signal to determine the direction: if B is `HIGH`, the rotation is clockwise, and the selected row is incremented (moves up); if B is `LOW`, the rotation is counterclockwise, and the selected row is decremented (moves down).

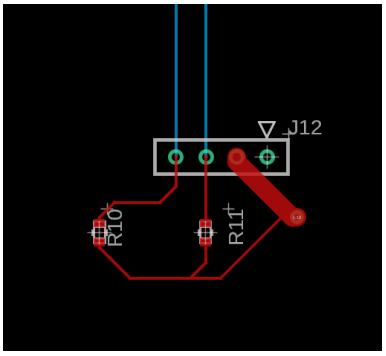
The `millis()` function is used to track elapsed time and introduce a delay (`rotationDelay`) between checks, preventing false or multiple detections of the same rotation step.

```
if ((millis() - lastUpdate) > rotationDelay) {
  if (sigA != lastSigA && sigA == LOW) {
        if (sigB == HIGH) {
          selectedRow = (selectedRow+1) % 4;
        } else {
          selectedRow = (selectedRow-1+4) % 4;
        }
        updateDisplay();
  }
  lastUpdate = millis();
}
```

B. OLED Display



OLED Display Schematic



OLED Display PCB Layout

The 128x64 pixel OLED display shows a menu to allow users to go through different g-code options. It is connected to the ATmega328P microcontroller using the I2C protocol (SDA and SCL pins). The display is controlled using the `Adafruit_GFX` and `Adafruit_SSD1306` libraries, which make it easier to draw graphics and text as needed [1].

When the device starts, it shows predefined g-codes to be selected. Turning the encoder moves the selection up or down. The currently selected row is highlighted with a black rectangle, making it easy to see which design or selection is chosen.

The UI design is drawn using `drawBitmap()` and updated regularly:

```
display.clearDisplay();
display.drawBitmap(0, 0, bitmaps[bitIndex],
MYBITMAP_WIDTH, SCREEN_HEIGHT, SSD1306_WHITE);
display.display();
```

C. Menu Selection

Once a design is selected using the encoder button, the system updates the selected index (`bitIndex`) and displays a print progress on the screen using bitmaps. These bitmaps are drawn using the `drawBitmap()` function.

The menu works as follows:

a) The user rotates the knob to scroll through four rows (g-code or menu options depending on the page).
b) Pressing the knob allows selection and updates the display.
c) A black rectangle shows which row is currently selected.

This is how the black rectangle selection is drawn over that area:

```
int rowTop = selectedRow * 16;
display.drawLine(0, rowTop, 127, rowTop,
SSD1306_BLACK);
display.drawLine(0, rowTop + 15, 127, rowTop +
15, SSD1306_BLACK);
```

Explanation of values:

a) `selectedRow * 16` calculates the vertical position (in pixels) of the top of the selected row. Since the display is 64 pixels tall and there are 4 rows, each row is 16 pixels high.

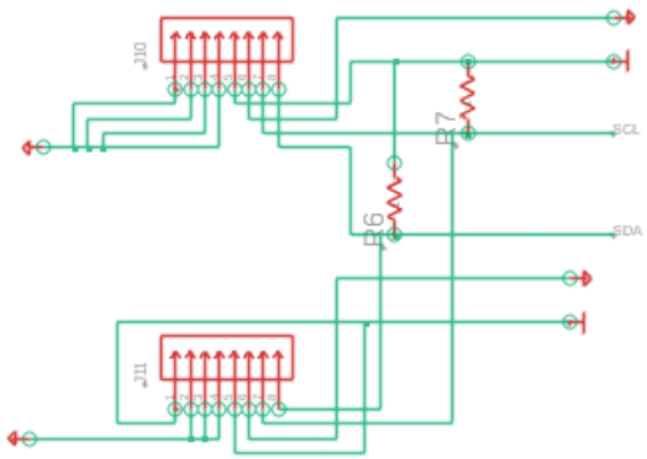
b) `127` is the maximum X-coordinate on a 128-pixel-wide screen (0 to 127).

c) `rowTop + 15` marks the bottom edge of the rectangle. The height of each row is 16 pixels, and since coordinates start from 0, the last pixel is at `rowTop + 15`.

This helps users clearly see which design is currently selected.

VIII. EEPROM Storage System.

To overcome the limited onboard memory of the ATmega328P, the 2D-Plotter employs two external EEPROMs (AT24C256) for expanded storage capacity. These EEPROMs are dedicated to storing G-code files, which define the precise motion paths for the plotter's stepper motors. This system enables the retention of multiple plot designs that can be selected and executed by the user without reprogramming the microcontroller.



A. EEPROM Configuration

The EEPROMs communicate with the ATmega328P via the I²C protocol, utilizing device addresses 0x50 and 0x51. Each provides 32 kB of non-volatile memory, offering a combined total of 64 kB for persistent data storage. The SDA and SCL lines are connected through 10 kΩ pull-up resistors to maintain signal integrity.

Each EEPROM supports 64-byte page writes. To ensure data integrity, write operations are performed in aligned 64-byte chunks, with a mandatory delay to accommodate the EEPROM's internal write cycle timing.

Example code snippet to write a 64-byte G-code block to EEPROM:

```
#define EEPROM_ADDR1 0x50
#define GCODE_START_ADDR 0x1000

uint8_t gcodeChunk[64] = { 'G', '0', '1', ' ',
'X', '1', '0', ' ', 'Y', '2', '0', '\n', /*
... */ };

void writeGcodeChunk() {
  Wire.beginTransmission(EEPROM_ADDR1);
  Wire.write((GCODE_START_ADDR >> 8) & 0xFF);
  Wire.write(GCODE_START_ADDR & 0xFF);
  for (uint8_t i = 0; i < 64; i++) {
    Wire.write(gcodeChunk[i]);
  }
  Wire.endTransmission();
  delay(5);
}
```

This process is iteratively executed to write entire G-code files into the EEPROM memory space.

B. Python-Driven File Management System

To streamline the process of uploading and managing G-code files, a Python-based graphical user interface (GUI) was developed. This system enables intuitive control over EEPROM file storage through Serial communication with the Arduino.

Key Features of the File Management System:

a) Chunked Uploads: Files are divided into 64-byte chunks to match EEPROM page size. Each chunk is transmitted with synchronized handshakes, ensuring data integrity.

b) Automatic File Indexing: New uploads are automatically assigned unique file numbers and mapped to available memory addresses.

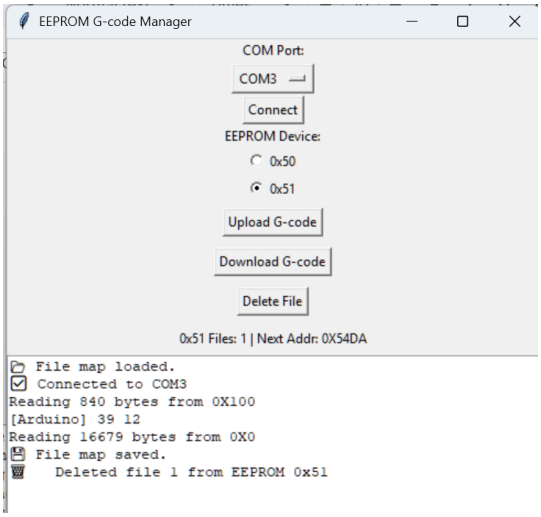
c) Multi-EEPROM Support: Users can select between two EEPROM devices (0x50 and 0x51) for file operations.

d) Persistent File Map: File information (name, EEPROM, address, size) is saved locally, allowing file records to persist between sessions.

e) Overwrite & Delete Functions: Existing files can be replaced or removed, updating the file map dynamically.

f) Download Capability: Stored G-code files can be read back from EEPROM, reconstructed, and displayed in the GUI for verification.

The GUI displays current file mappings, available EEPROM space, and real-time upload/download status, offering a user-friendly interface for memory management



EEPROM G-code Manager GUI

This integration simplifies the workflow for managing multiple G-code designs, providing clear visual feedback and robust file handling.

Upon uploading, the Python script divides G-code files into 64-byte chunks, transmitting each sequentially while awaiting confirmation from the microcontroller. Each file is assigned a unique identifier and mapped to its start address in EEPROM.

Example workflow:

1) Upload File: Python script initiates file upload, assigns file number, splits into chunks, and performs synchronized writes.
2) Download File: User selects EEPROM and file number. The system reads back the stored chunks, reconstructing the original G-code.
3) Delete File: The selected file is removed from the file map, freeing up its address range.
4) Persistent Map: The file map is saved locally, ensuring file indices remain consistent between sessions.

C. G-code Execution and Optimization

Once stored, the ATmega328P retrieves and parses G-code files in real time. Commands are buffered and converted into stepper motor signals for accurate plotting. Memory optimization strategies are employed to maximize storage efficiency, including:

a) Relative coordinate encoding (delta X/Y movements).
b) Removal of redundant command prefixes (e.g., omitting repetitive G01).
c) Exclusion of unused parameters (e.g., Z, F values when not applicable).

These methods significantly reduce the memory footprint of each design, enabling a larger library of patterns within the available EEPROM capacity.

IX. Conclusion

This article presents the current progress of our 2D plotter project, which is focused on building a functional drawing machine using unipolar stepper motors and an ATmega328P microcontroller. So far, we have implemented basic motor control, serial command parsing, EEPROM storage for G-code, and a working user interface using an OLED display and rotary encoder.

The system can interpret manual and preprogrammed drawing commands, move the pen accurately in two dimensions, and lift or lower the pen as needed. While many core features are complete, further work is planned to improve G-code support, streamline drawing performance, and expand UI functionality.

This write-up serves as a documentation of our current implementation and a reference for future development stages.

Acknowledgments

Angel Barrera, John San-Juan, and Ranbir Saini thank the authors of the open-source 2D plotter project published on [https://www.instructables.com/Arduino-Mini-CNC-Plotter] for providing publicly available 3D-printable designs and implementation guidance. The reference designs used in this article played an important role in helping the team rapidly prototype the mechanical components of the plotter. Their contribution provided a strong foundation during the early development stages of the project.

References

[1] L. P. Lam and P. D. K. Nagle, “Adafruit SSD1306 OLED display library,” Adafruit Industries, New York, NY, USA. [Online]. Available: https://github.com/adafruit/Adafruit_SSD1306
[2] Arduino Forum, “Reading rotary encoder with interrupts and debouncing,” Nov. 2022. [Online]. Available: https://forum.arduino.cc/
[3] Arduino, "Rotary Encoder with Arduino," [Online]. Available: https://www.arduino.cc/en/Tutorial/RotaryEncoder. [Accessed: Apr. 11, 2025].
[4] R. S. Malik, “Arduino Mini CNC Plotter,” Instructables. [Online]. Available: https://www.instructables.com/Arduino-Mini-CNC-Plotter/. [Accessed: Apr. 12, 2025].

Angel Barrera, photograph and biography not available at the time of publication

John San Juan, photograph and biography not available at the time of publication.

Ranbir Saini, photograph and biography not available at the time of publication.